

ASSESSMENT TOOL

SOLUTION

Question 1: Water Jug Problem

Problem Statement

A person has two jugs with capacities of **4 gallons** and **3 gallons**. Neither jug has measurement markings. The objective is to obtain **exactly 2 gallons of water in the 4-gallon jug** using only the available operations: filling a jug from the pump, emptying a jug, and pouring water from one jug to another.

Problem Analysis

Initial State

- 4-gallon jug = 0 gallons
- 3-gallon jug = 0 gallons

State:

(0,0)

Goal State

The required final state is

(2,0)

meaning:

- 4-gallon jug contains 2 gallons
- 3-gallon jug is empty

Constraints

- No measuring scale is available.
- Only filling, pouring, and emptying operations are allowed.

AI Solution

This problem is solved using **State Space Search**.

Each water distribution is considered as a **state**.

Example:

(3,1)

means

- 4-gallon jug contains 3 gallons
- 3-gallon jug contains 1 gallon

The search algorithm explores different states until the goal state is reached.

Step-by-Step Solution

Step State (4,3) Action

Start	(0,0)	Initial State
1	(0,3)	Fill 3-gallon jug
2	(3,0)	Pour into 4-gallon jug
3	(3,3)	Fill 3-gallon jug
4	(4,2)	Pour into 4-gallon jug until full
5	(0,2)	Empty 4-gallon jug
6	(2,0)	Pour remaining water into 4-gallon jug

Result

The goal state

(2,0)

is successfully achieved.

Conclusion

The Water Jug problem demonstrates how **State Space Search** systematically explores possible states to reach the desired goal without using any measuring device.

Question 2: Mars Rover Intelligent Agent

Problem Statement

NASA's Mars Rover is required to autonomously explore the surface of Mars, collect soil and rock samples, avoid obstacles, and transmit scientific data back to Earth. The challenge is that human operators cannot control the rover in real time due to communication delays. Therefore, the rover must make intelligent decisions on its own.

Problem Analysis

The rover operates in an unknown environment where it must:

- Detect obstacles
- Identify interesting rocks
- Collect samples
- Navigate safely
- Manage battery power

AI Solution

A **Model-Based Goal-Based Agent** is suitable because it:

- Receives sensor information.
- Maintains an internal model of the environment.
- Plans actions to achieve mission goals.
- Updates decisions based on new observations.

Working

Inputs (Percepts)

- Cameras
- Temperature sensor

- Rock sensor
- Soil sensor
- Navigation sensor
- Battery sensor



Processing

The AI system:

- Analyzes terrain
- Detects obstacles
- Plans safest route
- Selects rock samples
- Monitors battery



Actions

- Move
- Turn
- Drill
- Collect sample
- Analyze sample
- Send data
- Recharge

Result

The rover safely explores Mars, collects scientific samples, and sends useful information to Earth with minimal human intervention.

Conclusion

The Mars Rover is an excellent example of an intelligent autonomous agent capable of perception, reasoning, planning, and decision-making in a dynamic environment.

Question 3: 8-Queens Problem

Problem Statement

The **8-Queens Problem** requires placing **8 queens** on an **8 × 8 chessboard** such that no queen attacks another queen. A queen can attack horizontally, vertically, and diagonally. The objective is to find a valid arrangement where all queens are safe.

Problem Analysis

Initial State

- An empty 8 × 8 chessboard.
- No queens are placed.

Goal State

- Place all 8 queens on the chessboard.
- No two queens share the same:
 - Row
 - Column
 - Diagonal

Constraints

- Only one queen can be placed in each row.
- A queen cannot attack another queen.
- Every queen must be placed in a safe position.

AI Solution

The problem is solved using **Backtracking Search**, a form of **Depth-First Search (DFS)**.

Why Backtracking?

- It places queens one row at a time.

- If a conflict occurs, it removes the last placed queen.
- It tries another safe position until a valid solution is found.

This avoids checking all possible arrangements and makes the search more efficient.

Step-by-Step Working

Step 1

Start with an empty chessboard.

Step 2

Place the first queen in the first row.

Step 3

Move to the next row and place another queen in a safe column.

Step 4

Continue placing queens row by row.

Step 5

If no safe position is available in a row:

- Remove the previous queen.
- Try another column.

Step 6

Repeat until all eight queens are placed safely.

Problem Formulation

Component Description

Initial State Empty chessboard

State Board with queens placed safely

Operators Place or move a queen

Goal Test Eight queens placed without attacking each other

Path Cost Each queen placement costs 1

Result

The algorithm successfully finds a chessboard arrangement where all eight queens are placed without any conflicts.

Conclusion

The **8-Queens Problem** demonstrates how **Backtracking Search** efficiently solves constraint satisfaction problems by eliminating invalid solutions early and exploring only safe configurations.

Question 4: OLA Cab Booking Mobile Application

Problem Statement

A customer wants to travel from one location to another using the **OLA Cab Booking Application**. The customer can choose different cab types such as **Mini, Micro, Sedan, Shared, or Prime** based on comfort and budget. The system must identify the appropriate AI problem-solving agent and describe how the booking process works.

Problem Analysis

Inputs

- Pickup location
- Destination
- Cab type
- Customer preferences

Expected Output

- Assign the most suitable cab.
- Display fare and estimated arrival time.
- Complete the booking successfully.

Constraints

- Cab availability
- Distance
- Traffic conditions

AI Solution

Type of Agent

A **Goal-Based Agent** is the most suitable.

Reason

The customer's goal is to **reach the destination safely and efficiently**. The agent selects actions that help achieve this goal by considering:

- Cab availability
- Customer preferences
- Route
- Estimated travel time

Pseudocode

START

Input pickup location

Input destination

Input preferred cab type

Search for available drivers

IF cab is available THEN

 Display fare estimate

 Confirm booking

 Assign nearest driver

 Notify customer

```
    Start trip
    Reach destination
    Generate final bill
    Complete payment
ELSE
    Display "No cab available"
ENDIF

STOP
```

Working Process

1. Customer enters pickup and destination.
2. The system searches for nearby drivers.
3. It calculates the estimated fare.
4. The customer confirms the booking.
5. The nearest driver is assigned.
6. The driver picks up the customer.
7. The customer reaches the destination.
8. Payment is completed.

Result

The Goal-Based Agent successfully identifies the customer's objective and selects the best available cab to complete the journey.

Conclusion

The OLA booking system is an example of a **Goal-Based Intelligent Agent** because it makes decisions that help the customer reach the desired destination while considering preferences, availability, and travel conditions.

Question 5: Uniform Cost Search (UCS)

Problem Statement

A logistics company operates a network of warehouses connected by routes with different travel costs. The objective is to determine the **least-cost path** from the starting warehouse **S** to the destination warehouse **G** using the **Uniform Cost Search (UCS)** algorithm.

Problem Analysis

Initial State

- Start warehouse **S**.

Goal State

- Reach destination warehouse **G**.

Constraints

- Every route has a travel cost.
- The algorithm must always choose the path with the minimum total cost.

AI Solution

The **Uniform Cost Search (UCS)** algorithm is used because it always expands the node with the **lowest cumulative path cost**.

Unlike Breadth-First Search, UCS considers the travel cost of each route before selecting the next node.

Algorithm Working

Step 1

Insert the starting node **S** into a priority queue with cost **0**.

Step 2

Remove the node with the smallest cumulative cost.

Step 3

Check whether the node is the goal (**G**).

Step 4

If it is not the goal:

- Expand all neighboring nodes.
- Calculate the cumulative cost for each neighbor.
- Insert them into the priority queue.

Step 5

Repeat the process until the destination node is reached.

Pseudocode

START

Insert Start Node (S) into Priority Queue

WHILE Queue is not Empty

 Remove Node with Lowest Cost

 IF Node = Goal THEN

 Return Path and Total Cost

 ENDIF

 Expand Neighboring Nodes

 Calculate New Cost

 Insert into Priority Queue

END WHILE

STOP

Advantages

- Finds the least-cost path.
- Guarantees an optimal solution when all edge costs are non-negative.
- Suitable for route planning and logistics.

Applications

- GPS navigation systems
- Delivery route optimization
- Robot navigation
- Airline route planning
- Supply chain management

Result

Uniform Cost Search identifies the **minimum-cost route** from warehouse **S** to warehouse **G**, ensuring that transportation is completed efficiently with the lowest possible travel cost.

Conclusion

Uniform Cost Search is an optimal search algorithm for weighted graphs because it always expands the path with the lowest cumulative cost. This makes it highly suitable for logistics and transportation problems where minimizing cost and time is essential.